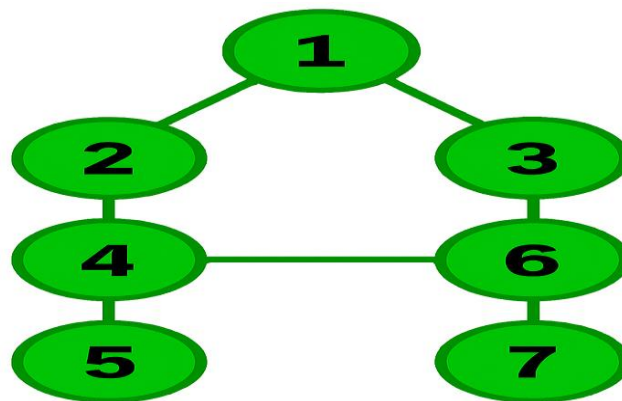


Tree Data Structures (From Basics to Advanced)

Introduction to Trees

A Tree is a hierarchical data structure that simulates a tree structure with a root and branches. It is a widely used non-linear data structure where each element, called a node, is connected by edges to other nodes in a parent-child relationship. The topmost node is known as the root, and nodes without children are called leaves. Unlike linear structures, trees allow efficient organization and retrieval of data in hierarchical formats. They are commonly used in various applications such as databases, file systems, and decision-making processes. Popular types include binary trees, binary search trees, AVL trees, and heaps.



What is the Use of Recursion in Tree Data Structures?

Recursion is one of the most powerful tools when working with tree data structures, especially in Java and other object-oriented languages.

It simplifies traversal, insertion, and deletion by mirroring the tree's hierarchical nature.

Why Recursion Works So Well with Trees

Trees are inherently recursive structures:

- Each node contains references to its children (subtrees)
- So the whole tree is made of smaller trees (left/right)
- Recursion allows you to naturally traverse and process each subtree

USE CASE	DESCRIPTION
TRAVERSAL	Visit nodes in in-order, pre-order, post-order, etc.
SEARCH	Find an element in a binary search tree (BST)
INSERT/DELETE	Add or remove nodes in BSTs
HEIGHT/DEPTH CALCULATION	Compute how tall the tree is
PATH-BASED PROBLEMS	E.g., max path sum, root-to-leaf paths
STRUCTURAL COMPARISON	Check if two trees are the same
TRANSFORMATION	Mirror, flatten, clone trees

Use of Recursion in Trees

Benefits of Using Recursion

Advantage	Explanation
Simplifies code	Avoids boilerplate of stack/loop logic
Matches tree structure naturally	Each node is a mini-tree
Easy to reason about	Break problem into smaller parts
Elegant solutions	Often much cleaner than iteration

When to Be Careful

- Stack overflow: Deep trees (like linked lists) can cause too much recursion.
- In Java, recursion depth is limited (~10,000 calls by default).
- Use iteration with a stack if the tree is very deep or if memory is critical.

Standard Approach to Solve Tree Problems

1. Understand the Type of Tree

Determine what kind of tree you are working with:

- Binary Tree – each node has at most two children
- Binary Search Tree (BST) – $\text{left} < \text{root} < \text{right}$
- Balanced Tree – height is controlled (e.g., AVL, Red-Black Tree)
- N-ary Tree – nodes can have more than two children
- Trie – for string/prefix matching

This helps define the rules of traversal and insertion

2. How to Define a Tree in Java:

We usually define a Tree Node using a class, and then manage the overall structure using another class (like BinaryTree).

Step 1: Define the Node Class

```
class TreeNode {  
    int data;  
    TreeNode left;  
    TreeNode right;  
  
    TreeNode(int value) {  
        data = value;  
        left = null;    // initially no children  
        right = null;  
    }  
}
```

Why Use a Class to Define a Tree Node?

1. **Encapsulation:** A class allows bundling of data (value) and references (left/right children) in one unit.
2. **Reusability:** The `TreeNode` class can be reused across different types of trees (binary, BST, AVL, etc.).
3. **Object-Oriented Design:** It fits Java's object-oriented paradigm, making it easy to manage complex operations like traversal, insertion, deletion, etc.
4. **Memory Efficiency:** Each node is dynamically allocated in memory as a separate object, avoiding unnecessary memory for unused branches.
5. **Scalability:** You can easily extend the class to support extra features (like parent pointers, balance factors, colors for Red-Black Trees, etc.).

Step 2: Define the Tree Class

```
public class GameTree {  
    TreeNode root;  
  
    public GameTree() {  
        root = null;           // initially, tree is empty  
    }  
  
    // Custom method to build the tree manually  
    public void buildGameLevelTree() {  
        root = new TreeNode("Game Start");  
        root.left = new TreeNode("Level 1");  
        root.right = new TreeNode("Level 2");  
  
        root.left.left = new TreeNode("Boss 1");  
        root.left.right = new TreeNode("Mini-Game A");  
  
        root.right.left = new TreeNode("Mini-Game B");  
        root.right.right = new TreeNode("Boss 2");  
    }  
}
```

Or for N-ary trees:

```
class Node {  
    int val;  
    List<Node> children;  
}
```

3. Decide on the Traversal Type

Traversal is key to solving most tree problems. Choose based on the problem:

Traversal Type	Description	Use Cases
Pre-order	Root → Left → Right	Copy tree, prefix expression
In-order	Left → Root → Right	BST sorted order
Post-order	Left → Right → Root	Delete tree, postfix expression
Level-order	Breadth-first traversal (BFS)	Shortest path, tree levels

You can implement these using recursion or stacks/queues.

4. Choose Recursion or Iteration

- Most tree problems are easier with recursion (natural for tree structure).
- Some problems require iteration with stack/queue for memory control.

Example (In-order):

```
void inorder(TreeNode root) {  
    if (root == null) return;  
    inorder(root.left);  
    System.out.print(root.data + " ");  
    inorder(root.right);  
}
```

5. Think in Terms of Subproblems (Divide and Conquer)

Many tree problems can be solved using recursive thinking:

Solve the left subtree → Solve the right subtree → Combine results

Example: Height of a Tree

```
int height(TreeNode root) {  
    if (root == null) return 0;  
    return 1 + Math.max(height(root.left), height(root.right));  
}
```

6. Use Helper Functions or Global Variables (if needed)

Some problems (like finding the diameter, max path sum) may require you to keep track of global state during recursion.

```
int max = 0;  
int diameter(TreeNode root) {  
    dfs(root);  
    return max;  
}  
  
int dfs(TreeNode node) {  
    if (node == null) return 0;  
    int left = dfs(node.left);  
    int right = dfs(node.right);  
    max = Math.max(max, left + right);  
    return 1 + Math.max(left, right);  
}
```

7. Edge Cases

Always think about:

- Empty tree (root == null)
- Single-node tree
- Unbalanced tree (deep on one side)
- Duplicate values (if allowed)

Variable State in a Tree (Initially):

When you first define a Tree structure in Java, several key variables are involved — mainly the root node and possibly its children. Let's go step by step.

Before Tree is Constructed

☒ **Declaration (But Not Instantiation):**

```
BinaryTree tree = new BinaryTree();
```

At this point:

- A BinaryTree object is created.
- The root variable inside BinaryTree is still null.

Memory State:

Variable	Value	Description
tree	Object	Reference to a BinaryTree object
tree.root	null	Tree has not been constructed yet

After Creating the Root Node:

```
tree.root = new TreeNode(1);
```

Now:

- The root node exists and stores the value 1.
- root.left and root.right are still null.

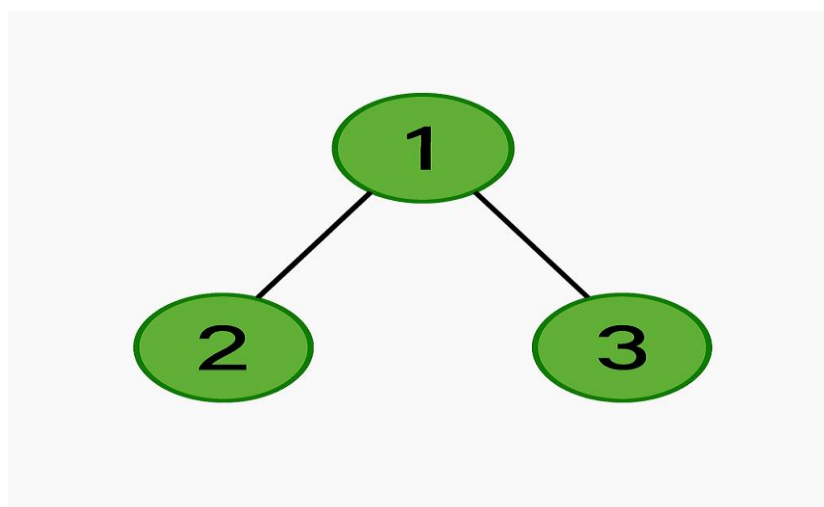
Memory State:

Variable	Value	Description
tree.root	TreeNode(1)	Root node created
tree.root.left	null	No left child yet
tree.root.right	null	No right child yet

After Adding Children:

```
tree.root.left = new TreeNode(2);  
tree.root.right = new TreeNode(3);
```

Memory Tree View:



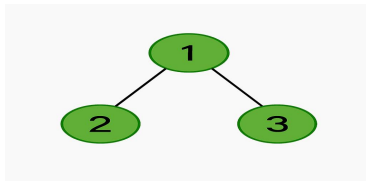
Variable States:

Variable	Value	Description
tree.root	TreeNode(1)	Root node
tree.root.left	TreeNode(2)	Left child of root
tree.root.right	TreeNode(3)	Right child of root
tree.root.left.left	null	Not yet defined
tree.root.left.right	null	Not yet defined

Basic Tree Terminologies

- **Hierarchical Structure:** A non-linear structure where nodes are arranged in levels, creating a tree-like formation with parent-child relationships.
- **Root Node:** The topmost node in a tree that does not have any parent and serves as the origin point.

Example Tree:



In this example, 1 is the root node.

- **Edges:** The connections or links between two nodes, representing the relationship between parent and child within the tree.

In a tree with n nodes, there are exactly $n - 1$ edges.

In the above tree:

Edge between 1 and 2

Edge between 1 and 3

- **Child Nodes:** Nodes directly connected below another node, representing a downward connection from the parent in the tree structure.

2 and 3 are children of 1

- **Parent Node:** A node with one or more child nodes directly beneath it, creating a branching structure in the hierarchy.

1 is the parent of 2 and 3 in above diagram

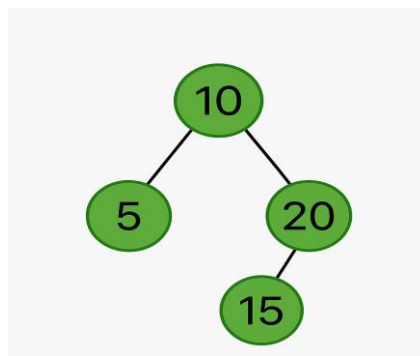
- **Leaf Node:** A terminal node that does not have any children and typically represents the end point of a path.

Nodes 2 and 3 are leaf nodes in above diagram.

```
boolean isLeaf(TreeNode node) {  
    return node.left == null && node.right == null;  
}
```

- **Height:** The length of the longest path from a node down to a leaf, indicating the depth of its subtree.

```
int height(TreeNode node) {
    if (node == null) return -1; // use 0 if height of leaf is 0
    return 1 + Math.max(height(node.left), height(node.right));
}
```



Height of 10 = 2

Height of 20 = 1

- **Depth:** The number of edges from the root node to a given node, indicating that node's level in the tree.

```
int depth(TreeNode root, TreeNode target, int level) {
    if (root == null) return -1;
    if (root == target) return level;
    int left = depth(root.left, target, level + 1);
    if (left != -1) return left;
    return depth(root.right, target, level + 1);
}
```

The root node itself has a depth of 0.

Depth of each node:

- **10** → Depth **0** (root)
 - **5** → Depth **1** (one edge from 10)
 - **20** → Depth **1** (one edge from 10)
 - **15** → Depth **2** (edge from 10 → 20 → 15)
-
- **Binary Tree:** A special tree in which each node has at most two children, usually referred to as left and right.
 - **Subtree:** Any branch of the tree that starts from a node and includes all of its descendants and connecting edges.



- **Siblings:** Nodes that share the same parent node in the tree, appearing at the same level within the structure.

2 and 3 are siblings.

- **Level:** Represents the distance of a node from the root, counted by how many connections or edges it takes.

For example in above diagram:

Level 0: 10

Level 1: 2, 3

- **Degree of Node:** Refers to the number of immediate children that a node has connected directly beneath it in the tree.
- **Balanced Tree:** A tree where the heights of left and right subtrees for every node differ by no more than one.
- **Full Binary Tree:** A binary tree in which every node has either zero or exactly two children, never just one child.

- **Complete Tree:** All levels are fully filled, except possibly the last level, which is filled from left to right.
- **Skewed Tree:** A tree where all nodes lean to one side, forming a linear structure either to the left or right.
- **Traversal:** The process of visiting every node in a tree in a specific order such as inorder, preorder, or postorder.
- **Ancestor Node:** Any node on the path from a given node back to the root, representing its lineage in the hierarchy.

Descendant Node: A node that falls under another node, including children, grandchildren, and all further nested connections below.

Types of Trees

Trees can be categorized based on their structure, properties, and the number of children per node. Let's explore key tree types that are fundamental in DSA and interviews.

1. General Tree

- A general tree allows each node to have any number of children.
- It's commonly used in hierarchical data like file systems, organization charts.

Properties:

- No limit on the number of children
- Flexible structure

```
import java.util.ArrayList;
import java.util.List;

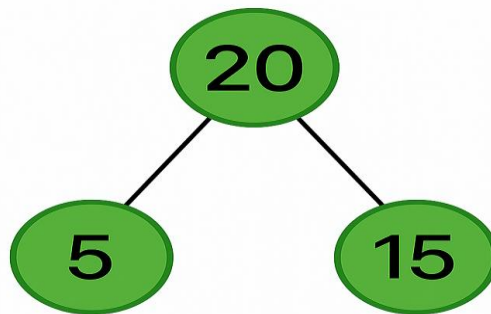
class GeneralTreeNode {
    int data;
    List<GeneralTreeNode> children;

    public GeneralTreeNode(int data) {
        this.data = data;
        this.children = new ArrayList<>();
    }
}
```

2. Binary Tree

- A tree where each node has at most two children: left and right.
- It is the basis for most tree problems in DSA.

binary tree



```
class BinaryTreeNode {  
    int data;  
    BinaryTreeNode left, right;  
  
    public BinaryTreeNode(int data) {  
        this.data = data;  
    }  
}  
  
public void addChild(BinaryTreeNode child) {  
    children.add(child);  
}  
}
```

3. Full Binary Tree

- Every node has **0 or 2 children**.
- No node has only one child.

```

boolean isFullTree(BinaryTreeNode node) {
    if (node == null) return true;
    if ((node.left == null && node.right != null) ||
        (node.left != null && node.right == null)) {
        return false;
    }
    return isFullTree(node.left) && isFullTree(node.right);
}

```

4. Perfect Binary Tree

- All internal nodes have 2 children
- All leaf nodes are at the **same level**

```

int depth(BinaryTreeNode node) {
    int d = 0;
    while (node != null) {
        d++;
        node = node.left;
    }
    return d;
}

boolean isPerfectTree(BinaryTreeNode root, int depth, int level) {
    if (root == null) return true;
    if (root.left == null && root.right == null)
        return depth == level + 1;
    if (root.left == null || root.right == null)
        return false;
    return isPerfectTree(root.left, depth, level + 1) &&
        isPerfectTree(root.right, depth, level + 1);
}

```


5. Complete Binary Tree

- All levels except possibly the last are completely filled
- Last level is filled from **left to right**

```
boolean isCompleteTree(BinaryTreeNode root) {  
    Queue<BinaryTreeNode> queue = new LinkedList<>();  
    queue.add(root);  
    boolean end = false;  
  
    while (!queue.isEmpty()) {  
        BinaryTreeNode node = queue.poll();  
        if (node == null) {  
            end = true;  
        } else {  
            if (end) return false;  
            queue.add(node.left);  
            queue.add(node.right);  
        }  
    }  
    return true;  
}
```

6. Balanced Binary Tree

- A binary tree is balanced if **height difference** between left and right subtrees for every node is ≤ 1 .
- Improves performance for insert, delete, search.

```

int checkHeight(BinaryTreeNode node) {
    if (node == null) return 0;

    int left = checkHeight(node.left);
    if (left == -1) return -1;

    int right = checkHeight(node.right);
    if (right == -1) return -1;

    if (Math.abs(left - right) > 1) return -1;

    return Math.max(left, right) + 1;
}

```

7. Binary Search Tree (BST)

- Binary Tree with left subtree values < node < right subtree values
- Used in efficient search and range queries

```

BinaryTreeNode insertBST(BinaryTreeNode root, int key) {
    if (root == null) return new BinaryTreeNode(key);
    if (key < root.data)
        root.left = insertBST(root.left, key);
    else
        root.right = insertBST(root.right, key);
    return root;
}

```

8. AVL Tree (Self-Balancing BST)

- An AVL Tree is a self-balancing Binary Search Tree where the height difference (balance factor) between the left and right subtrees of any node is at most 1, ensuring logarithmic height.
- It uses rotations Left-Left (LL), Right-Right (RR), Left-Right (LR), and Right-Left (RL) to restore balance after insertions or deletions that disturb the tree's height invariants.

9. Heap Tree

- A Heap Tree is a complete binary tree where all levels are fully filled except possibly the last, which is filled from left to right without gaps.
- It satisfies the heap property: in a Max Heap, each parent node is greater than or equal to its children; in a Min Heap, each parent node is less than or equal to its children.
- Follows the **heap property**:
 - Min Heap: $\text{parent} \leq \text{children}$
 - Max Heap: $\text{parent} \geq \text{children}$

10. Trie (Prefix Tree)

- A Trie (Prefix Tree) is a tree-based data structure that is highly efficient for string prefix searches, often used in autocomplete and dictionary implementations.
- In a Trie, each edge represents a character, and paths from the root to a node represent prefixes or complete words.

Tree Traversals

Tree Traversal refers to the process of visiting each node in a tree exactly once, in a specific order, to perform an operation (e.g., print, calculate, store).

There are two main categories of traversal:

1. Depth-First Traversal (DFS)

- Pre-order
- In-order
- Post-order

2. Breadth-First Traversal (BFS)

- Level-order

Key Points about Tree Traversals

1. Traversal means visiting every node once — order matters depending on the type.
2. DFS uses recursion or a stack, while BFS uses a queue.
3. In Binary Trees, DFS has 3 variations: inorder, preorder, postorder.
4. Traversal is used for printing, evaluating expressions, copying trees, etc.
5. Understanding traversal helps solve tree problems like serialization, height, diameter, etc.

Concepts & Types of Traversal

Let's go through each type in detail with illustrations and code.

1. Inorder Traversal (Left, Node, Right - LNR)

Concept:

- Visit left subtree
- Visit current node
- Visit right subtree

Application:

- Returns sorted order for Binary Search Trees

```
void inorder(TreeNode root) {  
    if (root == null) return;  
    inorder(root.left);  
    System.out.print(root.data + " ");  
    inorder(root.right);  
}
```

2. Preorder Traversal (Node, Left, Right - NLR)

Concept:

- Visit current node
- Visit left subtree
- Visit right subtree

Application:

- Used for copying trees, serialization

```
void preorder(TreeNode root) {  
    if (root == null) return;  
    System.out.print(root.data + " ");  
    preorder(root.left);  
    preorder(root.right);  
}
```

Output:

1 2 4 5 3 6

3. Postorder Traversal (Left, Right, Node - LRN)

Concept:

- Visit left subtree
- Visit right subtree
- Visit current node

Application:

- Used for deleting or freeing memory
- Used in expression evaluation

```
void postorder(TreeNode root) {  
    if (root == null) return;  
    postorder(root.left);  
    postorder(root.right);  
    System.out.print(root.data + " ");  
}
```

Output:

4 5 2 6 3 1

4. Level-Order Traversal (Breadth-First Search - BFS)

Level-Order Traversal is a tree traversal technique that visits nodes level by level from top to bottom and left to right, typically implemented using a queue and BFS strategy.

Concept:

- Visit nodes level by level from left to right
- Use a queue for implementation

Application:

- Shortest path in unweighted graphs
- Hierarchical processing (e.g., trees, org charts)
- LCA preprocessing (depth, binary lifting)
- Maze/grid problems (e.g., shortest path in 2D grid)
- Social networks (finding degrees of separation)

```

void levelOrder(TreeNode root) {
    if (root == null) return;

    Queue<TreeNode> queue = new LinkedList<>();
    queue.add(root);

    while (!queue.isEmpty()) {
        TreeNode node = queue.poll();
        System.out.print(node.data + " ");
        if (node.left != null) queue.add(node.left);
        if (node.right != null) queue.add(node.right);
    }
}

```

Output:

1 2 3 4 5 6

Iterative DFS using Stack (Preorder)

Depth-First Search is a graph/tree traversal algorithm that explores as far as possible along each branch before backtracking, typically implemented using recursion or an explicit stack.

- Used in topological sorting, cycle detection, and connected components.
- Efficient for deep structures, such as trees or mazes.
- Can be implemented recursively or using a stack.
- Time complexity: $O(V + E)$ for graphs (V = vertices, E = edges).

For interview scenarios where recursion is not allowed.

```
void iterativePreorder(TreeNode root) {  
    if (root == null) return;  
  
    Stack<TreeNode> stack = new Stack<>();  
    stack.push(root);  
  
    while (!stack.isEmpty()) {  
        TreeNode node = stack.pop();  
        System.out.print(node.data + " ");  
  
        if (node.right != null) stack.push(node.right);  
        if (node.left != null) stack.push(node.left);  
    }  
}
```

Notes:

Traversal Type	Order	Uses
Inorder (DFS)	L → Node → R	Sorting (BST)
Preorder (DFS)	Node → L → R	Tree copy, serialization
Postorder (DFS)	L → R → Node	Deletion, expression tree
Level-order (BFS)	Level by level	Shortest path, hierarchy

Binary Trees

A Binary Tree is a hierarchical data structure in which each node has at most two children, referred to as the left child and right child.

It is the basis for many specialized trees like Binary Search Tree, AVL, Heaps, etc.

Key Characteristics

1. Each node has ≤ 2 children.
2. Recursive structure: Each subtree is itself a binary tree.
3. Binary Trees can be used to represent expression trees, parsing, decision-making, etc.
4. Node connections follow a parent $\rightarrow$ left/right child format.
5. Tree depth and balance affect performance in operations like search and traversal.

Types of Binary Trees

Type	Description
Full Binary Tree	Every node has 0 or 2 children
Perfect Binary Tree	All internal nodes have two children and all leaves are at the same level
Complete Binary Tree	All levels completely filled except possibly the last, filled from left to right
Skewed Tree	All nodes have only one child (left or right)
Balanced Binary Tree	Height difference between left and right subtree ≤ 1
Degenerate Tree	Each parent has only one child — behaves like a linked list

Important Concepts & Operations

1. Tree Height

- The height of a tree is the number of edges on the longest path from root to leaf.
- Height of a single node = 0

```
int height(TreeNode root) {  
    if (root == null) return -1; // or 0 depending on convention  
    return 1 + Math.max(height(root.left), height(root.right));  
}
```

2. Size of Tree

- Total number of nodes in the tree.

```
int size(TreeNode root) {  
    if (root == null) return 0;  
    return 1 + size(root.left) + size(root.right);  
}
```

3. Leaf Nodes Count

```
int countLeaves(TreeNode root) {  
    if (root == null) return 0;  
    if (root.left == null && root.right == null) return 1;  
    return countLeaves(root.left) + countLeaves(root.right);  
}
```

4. Print All Nodes (Inorder Traversal)

```
void inorder(TreeNode root) {  
    if (root == null) return;  
    inorder(root.left);  
    System.out.print(root.data + " ");  
    inorder(root.right);  
}
```

Formulas and Properties

Let:

- **n** = total number of nodes
- **l** = number of leaf nodes
- **i** = number of internal nodes (nodes with at least one child)
- **h** = height of tree
- **m** = maximum number of nodes at level h

1. Maximum Nodes in Binary Tree

Max nodes in a binary tree of height $h = 2^{(h+1)} - 1$

2. Maximum Nodes at Level l

Max nodes at level $l = 2^l$

3. Minimum Height for n Nodes

$\text{minHeight} = \text{ceil}(\log_2(n+1)) - 1$

4. Number of Leaf Nodes (l) in Full Binary Tree

$$l = i + 1$$

5. Total Nodes in Perfect Binary Tree

$$n = 2^{(h+1)} - 1$$

Applications of Binary Trees

- **Expression Trees:** Represent algebraic expressions.
- **Hierarchical Data Models:** File systems, organization charts.
- **Binary Search Trees (BST):** Fast lookup, insert, delete.
- **Heaps:** Priority Queues, scheduling algorithms.
- **Syntax Parsing Trees:** Compilers and interpreters.

Binary Search Tree (BST)

A Binary Search Tree (BST) is a node-based binary tree data structure where each node has at most two children. The left subtree holds values less than the node's value, while the right subtree holds values greater. This ordering enables efficient searching, insertion, and deletion operations, typically with time complexity of **$O(\log n)$** for balanced trees. BSTs are widely used in computer science for implementing dynamic sets and lookup tables, making them fundamental to many algorithms and systems. However, performance can degrade to **$O(n)$** if the tree becomes unbalanced, resembling a linked list.

A Binary Search Tree (BST) is a binary tree that maintains a specific order:

For every node,

Left Subtree Nodes < Root < Right Subtree Nodes

This rule applies recursively to all nodes in the tree.

Key Properties of Binary Search Tree (BST)

- All values in the left subtree are smaller than the root node.
- All values in the right subtree are larger than the root node.
- No duplicate elements (in most standard BSTs).
- BST supports efficient **search**, **insert**, and **delete** operations.

Time Complexities

Operation	Best Case	Average Case	Worst Case
Search	$O(\log n)$	$O(\log n)$	$O(n)$
Insert	$O(\log n)$	$O(\log n)$	$O(n)$
Delete	$O(\log n)$	$O(\log n)$	$O(n)$

Types of BST

Type	Description
Standard BST	Follows basic BST property
Self-Balancing BSTs	Automatically balances height (e.g. AVL Tree, Red-Black Tree)
Threaded BST	Null pointers are used to store inorder successor
Treap	BST + Heap property
Splay Tree	Recently accessed nodes are moved to root

1. Insertion in BST

Concept

Recursively insert into left or right subtree based on comparison with current node.

```
TreeNode insert(TreeNode root, int key) {  
    if (root == null) return new TreeNode(key);  
  
    if (key < root.data)  
        root.left = insert(root.left, key);  
    else if (key > root.data)  
        root.right = insert(root.right, key);  
  
    return root;  
}
```

2. Search in BST

Concept

Use binary logic:

- If $\text{key} < \text{root}$, go left
- If $\text{key} > \text{root}$, go right
- If equal, found

```
boolean search(TreeNode root, int key) {  
    if (root == null) return false;  
    if (key == root.data) return true;  
    if (key < root.data)  
        return search(root.left, key);  
    else  
        return search(root.right, key);  
}
```

3. Inorder Traversal of BST (Sorted Order)

```
void inorder(TreeNode root) {  
    if (root != null) {  
        inorder(root.left);  
        System.out.print(root.data + " ");  
        inorder(root.right);  
    }  
}
```

Inorder traversal of BST gives elements in sorted order.

4. Find Minimum and Maximum

```
int findMin(TreeNode root) {  
    while (root.left != null)  
        root = root.left;  
    return root.data;  
}
```

```
int findMax(TreeNode root) {  
    while (root.right != null)  
        root = root.right;  
    return root.data;  
}
```

5. Delete Node from BST

Cases to Handle:

1. Node is a leaf → just remove it.
2. Node has one child → replace node with child.
3. Node has two children → replace with inorder successor or predecessor.


```

TreeNode delete(TreeNode root, int key) {
    if (root == null) return null;

    if (key < root.data) {
        root.left = delete(root.left, key);
    } else if (key > root.data) {
        root.right = delete(root.right, key);
    } else {
        // Node to delete found
        if (root.left == null) return root.right;
        if (root.right == null) return root.left;

        // Case: two children
        root.data = findMin(root.right);
        root.right = delete(root.right, root.data);
    }
    return root;
}

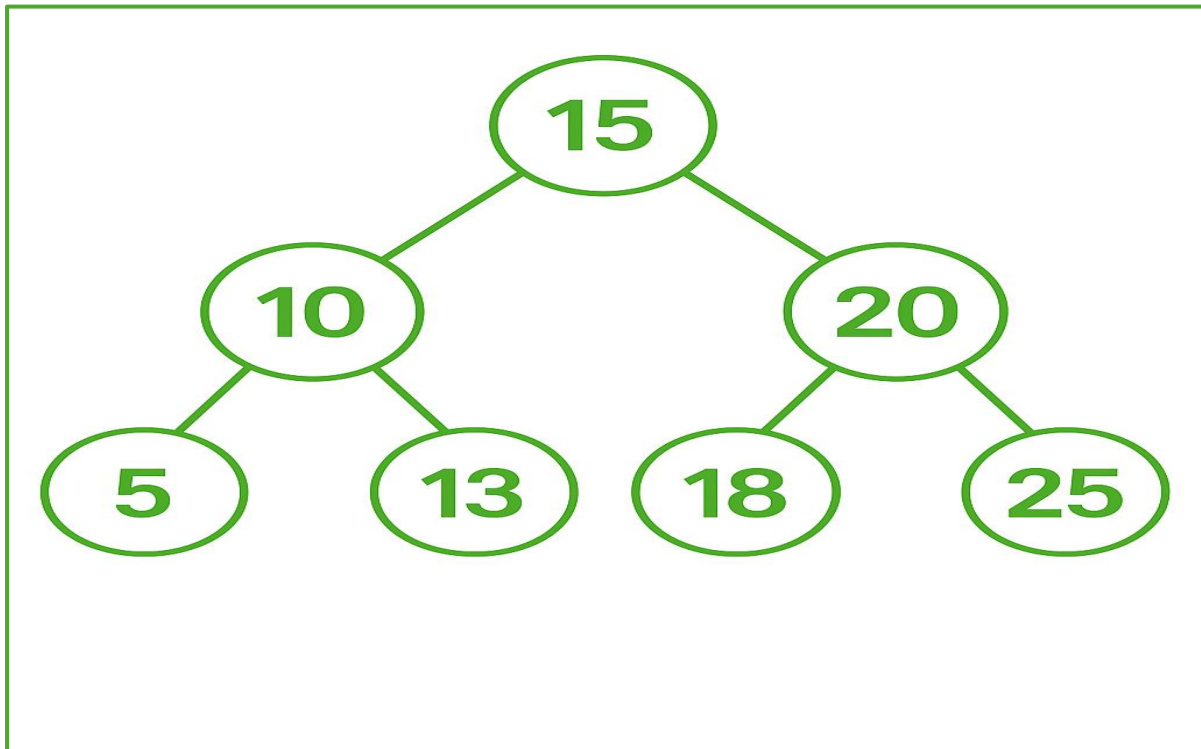
```

BST Formulas and Properties

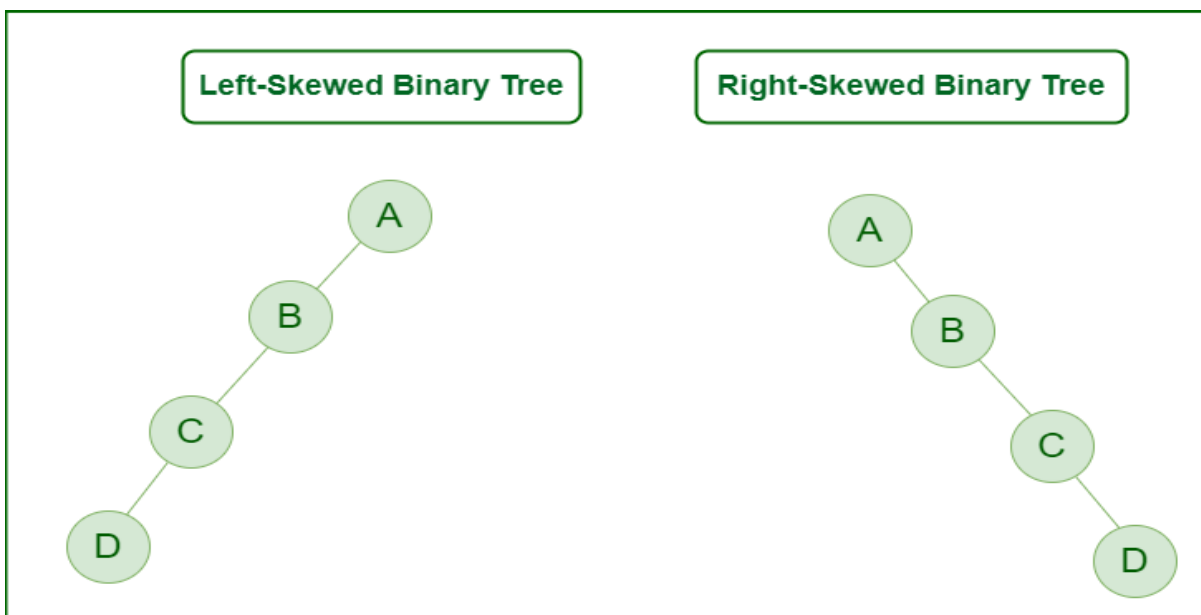
Property	Formula/Explanation
Min height (h)	$h = \text{floor}(\log_2 n)$ (for balanced BST)
Max height (h)	$h = n - 1$ (for skewed tree)
Inorder traversal	Yields sorted array
Nodes in subtree	$\text{size}(\text{root}) = 1 + \text{size}(\text{left}) + \text{size}(\text{right})$
Search/Insert/Delete (avg)	$O(\log n)$ (balanced)
Search/Insert/Delete (worst)	$O(n)$ (skewed)

Balanced vs Skewed BST

Balanced BST (Ideal Case):



Skewed BST (Worst Case, like Linked List):



In the skewed case, operations degrade to linear time $O(n)$.

Other Useful Operations

Count Nodes in BST

```
int countNodes(TreeNode root) {  
    if (root == null) return 0;  
    return 1 + countNodes(root.left) + countNodes(root.right);  
}
```

Height of BST

```
int height(TreeNode root) {  
    if (root == null) return -1;  
    return 1 + Math.max(height(root.left), height(root.right));  
}
```

BST Applications

- Binary Search Trees enable efficient searching, insertion, and deletion of elements in dynamically changing datasets.
- They are widely used in database indexing to quickly locate, insert, and manage records without full table scans.
- Autocomplete systems and spell checkers use BSTs to store and retrieve words based on prefixes in logarithmic time.
- Compilers use BSTs to implement symbol tables that manage identifiers, their scopes, and associated metadata.
- Operating systems utilize BSTs for priority scheduling by organizing processes based on their assigned priorities.
- Network routing algorithms use BSTs to store and search IP addresses for fast and optimized path resolution.

Binary Search Tree (BST) – Formulas & Properties

No.	Property	Formula / Value	Notes
1	Maximum Nodes (Balanced BST)	$2^{(h+1)} - 1$	Occurs in a perfectly balanced BST
2	Maximum Nodes at Level l	2^l	Level l starts from 0 (root level)
3	Minimum Height for n Nodes	$\text{ceil}(\log_2(n+1))-1$	Achieved when BST is fully balanced
4	Minimum Nodes for Height h	$h + 1$	Worst-case: skewed BST (like a linked list)
5	Maximum Height for n Nodes	$n - 1$	When all nodes are in a single path
6	Average Height (Random BST)	$\approx 1.39 * \log_2(n)$	On average for random insertions
7	Number of Unique BSTs with n Keys	$\text{Catalan}(n) = (2n)! / ((n+1)! * n!)$	Total structurally unique BSTs with n distinct keys
8	Total Nodes in Perfect BST	$2^{(h+1)} - 1$	All levels filled; same as balanced binary tree

Self-Balancing Trees

What is a Self-Balancing Tree?

A Self-Balancing Tree is a Binary Search Tree (BST) that automatically maintains its height close to $\log(n)$ after insertions and deletions, by applying rotations or restructuring.

These trees avoid becoming skewed (linear), ensuring all operations (insert, delete, search) stay efficient:

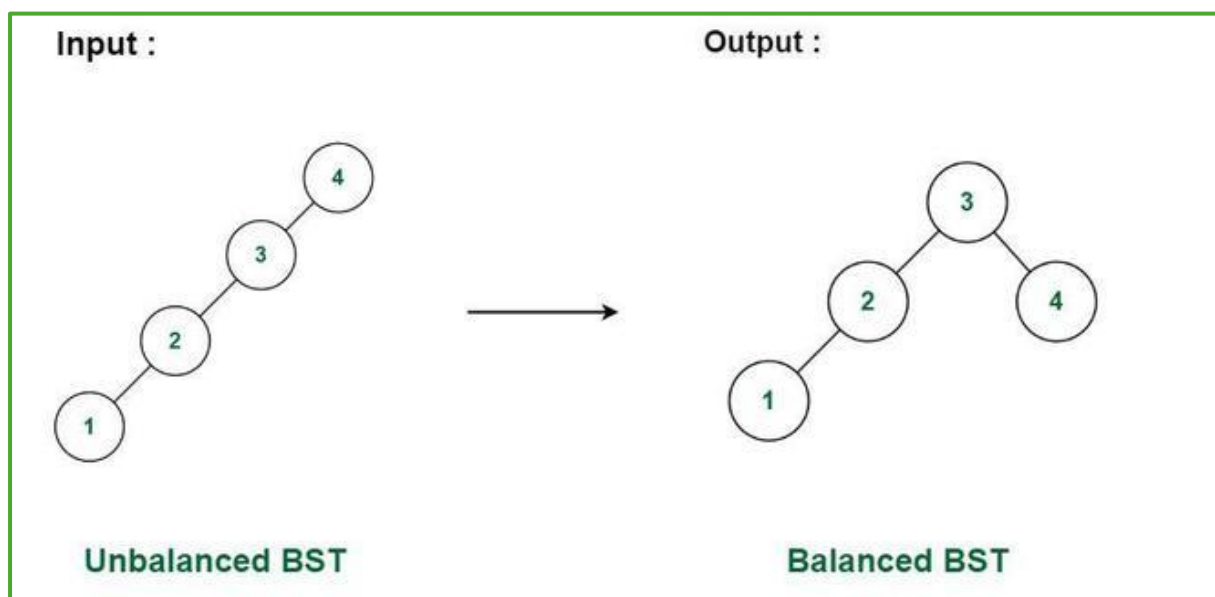
- **Time Complexity:** $O(\log n)$ for all operations

Why Are Self-Balancing Trees Needed?

Problem with Normal BST:

Insert: $1 \rightarrow 2 \rightarrow 3 \rightarrow 4$

Resulting BST:



Height = $n \rightarrow$ Degenerates into a **linked list**
Operations become $O(n)$

Self-Balancing BST:

Automatically restructures to:

Height $\approx \log n \rightarrow$ Operations remain **efficient**

Key Concepts in Self-Balancing Trees

Concept	Description
Rotation	Rearranging nodes to restore balance
Balance Factor	Height(left) – Height(right)
Height Invariant	Tree height always remains $O(\log n)$
Balance Rules	Vary by tree type (e.g., AVL vs RBT)

Types of Self-Balancing Trees

Tree Type	Balancing Method	Used In
AVL Tree	Height-based (strict)	Databases, indexing engines
Red-Black Tree	Color-based (relaxed)	Java TreeMap, C++ STL
Splay Tree	Move frequently used items to root	Caches, Zip Trees
Treap	Randomized heap + BST	Randomized data structures
Scapegoat Tree	Partial rebuilds	Memory-efficient BST variant
B-Trees / B+ Trees	Disk-aware, m-way balanced	Databases, File systems

1. AVL Tree (Strictly Balanced)

An AVL Tree is a self-balancing binary search tree (BST), where the balance factor (difference in height of left and right subtree) for every node is in the range $[-1, 0, 1]$.

Named after its inventors **Adelson-Velsky and Landis**, AVL trees automatically balance themselves after insertion and deletion operations using tree rotations.

$$|\text{height}(\text{left subtree}) - \text{height}(\text{right subtree})| \leq 1$$

- Maintains **balance factor** $[-1, 0, 1]$ at every node
- **Rotations after insert/delete**
- **Height balance is stricter → better search**

AVL Rotation Example:

```
if (balance > 1 && key < node.left.key)
    return rightRotate(node);
```

What is a Balanced Tree?

A Balanced Binary Tree is a tree where the height difference between left and right subtree for any node is minimized to maintain optimal time complexity for operations.

Why Balance Matters?

Type	Height (h)	Worst-case Time (search/insert/delete)
Unbalanced BST	$O(n)$	Linear, behaves like Linked List
Balanced BST	$O(\log n)$	Optimal, maintains performance

Why Self-Balancing Matters?

In a plain BST:

- Worst-case height = $O(n)$ → slow operations
- In an AVL tree: height = $O(\log n)$ → faster search, insert, and delete

So, AVL Trees guarantee logarithmic time complexity for fundamental operations.

Core AVL Tree Concepts

Concept	Description
Balance Factor (BF)	$BF = \text{height}(\text{left subtree}) - \text{height}(\text{right subtree})$
Balanced Node	BF is in $\{-1, 0, 1\}$
Unbalanced Node	BF is >1 or <-1 → requires rotation
Rotations	Used to maintain balance

Rotations in AVL Trees

Rotations restructure subtrees to regain balance. There are four cases:

1. Left-Left (LL) → Right Rotate
2. Right-Right (RR) → Left Rotate
3. Left-Right (LR) → Left Rotate + Right Rotate
4. Right-Left (RL) → Right Rotate + Left Rotate

Visual Example: LL Case

Before Insertion of 10:

```
  30
 /
20
/
10
```


→ Perform Right Rotation on 30

After:

```
    20
   / \
  10  30
```

AVL Tree Java Implementation

```
class AVLNode {
    int key, height;
    AVLNode left, right;

    public AVLNode(int key) {
        this.key = key;
        height = 1;
    }
}

int height(AVLNode node) {
    return node == null ? 0 : node.height;
}

int getBalance(AVLNode node) {
    return node == null ? 0 : height(node.left) - height(node.right);
}
```

Rotations

// Right Rotation

```
AVLNode rightRotate(AVLNode y) {  
    AVLNode x = y.left;  
    AVLNode T2 = x.right;  
  
    x.right = y;  
    y.left = T2;  
  
    y.height = Math.max(height(y.left), height(y.right)) + 1;  
    x.height = Math.max(height(x.left), height(x.right)) + 1;  
  
    return x;  
}
```

// Left Rotation

```
AVLNode leftRotate(AVLNode x) {  
    AVLNode y = x.right;  
    AVLNode T2 = y.left;  
  
    y.left = x;  
    x.right = T2;  
  
    x.height = Math.max(height(x.left), height(x.right)) + 1;  
    y.height = Math.max(height(y.left), height(y.right)) + 1;  
  
    return y;  
}
```

Insert with Rebalancing

```
AVLNode insert(AVLNode node, int key) {  
    if (node == null) return new AVLNode(key);  
  
    if (key < node.key)  
        node.left = insert(node.left, key);  
    else if (key > node.key)  
        node.right = insert(node.right, key);  
    else  
        return node; // no duplicates  
  
    node.height = 1 + Math.max(height(node.left), height(node.right));  
    int balance = getBalance(node);  
  
    // LL  
    if (balance > 1 && key < node.left.key)  
        return rightRotate(node);  
  
    // RR  
    if (balance < -1 && key > node.right.key)  
        return leftRotate(node);  
  
    // LR  
    if (balance > 1 && key > node.left.key) {  
        node.left = leftRotate(node.left);  
        return rightRotate(node);  
    }  
  
    // RL  
    if (balance < -1 && key < node.right.key) {  
        node.right = rightRotate(node.right);  
        return leftRotate(node);  
    }  
}
```

Inorder Traversal

```
void inorder(AVLNode root) {  
    if (root != null) {  
        inorder(root.left);  
        System.out.print(root.key + " ");  
        inorder(root.right);  
    }  
}
```

Formulas & Properties

Property	Formula
Balance Factor	$BF = \text{height}(\text{left}) - \text{height}(\text{right})$
Min Nodes in AVL of height h	$F(h) = F(h-1) + F(h-2) + 1$
Max Height of AVL Tree	$O(\log n)$
Time for Insertion/Deletion/Search	$O(\log n)$
Rotations	$O(1)$

AVL vs Other Trees

Feature	AVL Tree	Red-Black Tree	BST
Balance Strictness	High	Moderate	None
Rotation Count	Higher	Fewer	None
Performance	Better Search	Faster Insert/Delete	Worst-case slow
Use Case	Search-heavy systems	Frequent insert/delete	Simple cases

Key points:

- AVL Tree is a height-balanced BST, ideal for read-intensive systems.
- Guarantees logarithmic performance for all operations.
- Uses rotations to rebalance after insert/delete.
- AVL Trees are essential for intern-level DSA interviews and systems involving fast sorted data lookup.

Use When:

- Frequent searches
- Less frequent insert/delete

2. Red-Black Tree (Loosely Balanced)-Self-Balancing BST

What is a Red-Black Tree?

A Red-Black Tree is a type of self-balancing Binary Search Tree (BST) that ensures the tree remains approximately balanced, thus providing $O(\log n)$ time for insert, delete, and search operations.

Red-Black Tree Properties

To maintain balance, each node has an extra bit for color: RED or BLACK. The tree follows 5 core properties:

Property	Description
1.Node color	Each node is either RED or BLACK
2. Root color	The root is always BLACK
3. Red rule	No two RED nodes can be adjacent (no RED parent–RED child)
4.Black height	Every path from a node to its descendant NULL nodes has the same number of BLACK nodes
5. New node	New insertions are always RED (may be recolored later)

Why Red-Black Tree?

Feature	Description
Time Complexity	$O(\log n)$ for search, insert, delete
Rotations	Fewer than AVL in general
Used In	Java TreeMap, TreeSet, Linux Kernel, C++ STL map and set

Balancing in Red-Black Tree

Red-Black Trees balance themselves using recoloring and rotations:

Rotations

- Left Rotation
- Right Rotation
- Used when violations occur (e.g., RED-RED conflict)

Recoloring

- Change node colors to maintain red-black properties
- Often used when parent and uncle are RED

Red-Black Tree Node in Java

```
class RBNode {  
    int data;  
    RBNode left, right, parent;  
    boolean isRed;  
  
    public RBNode(int data) {  
        this.data = data;  
        this.isRed = true; // New nodes are red  
    }  
}
```

Red-Black Tree Insert Logic

Step-by-Step Approach:

1. Insert as in a regular BST.
2. Color the node RED.
3. Fix violations:
 - Case 1: Uncle is RED → Recolor parent & uncle → Grandparent might become unbalanced
 - Case 2: Uncle is BLACK or null → Perform rotations (LL, RR, LR, RL) + recoloring

4. Ensure root is always BLACK

Rotation Methods (Java)

```
// Left Rotation
void rotateLeft(RBNode x) {
    RBNode y = x.right;
    x.right = y.left;
    if (y.left != null)
        y.left.parent = x;

    y.parent = x.parent;
    if (x.parent == null)
        root = y;
    else if (x == x.parent.left)
        x.parent.left = y;
    else
        x.parent.right = y;

    y.left = x;
    x.parent = y;
}
```


// Right Rotation

```
void rotateRight(RBNode y) {  
    RBNode x = y.left;  
    y.left = x.right;  
    if (x.right != null)  
        x.right.parent = y;  
  
    x.parent = y.parent;  
    if (y.parent == null)  
        root = x;  
    else if (y == y.parent.left)  
        y.parent.left = x;  
    else  
        y.parent.right = x;  
  
    x.right = y;  
    y.parent = x;  
}
```

Fixing Red-Black Violations (Insert Fix)

```
void fixInsert(RBNode k) {  
    RBNode parent = null;  
    RBNode grandparent = null;  
  
    while (k != root && k.parent.isRed) {  
        parent = k.parent;  
        grandparent = parent.parent;  
  
        if (parent == grandparent.left) {  
            RBNode uncle = grandparent.right;
```

```

// Case 1: Uncle is RED → Recolor
if (uncle != null && uncle.isRed) {
    grandparent.isRed = true;
    parent.isRed = false;
    uncle.isRed = false;
    k = grandparent;
} else {
    // Case 2: Node is right child → rotateLeft
    if (k == parent.right) {
        k = parent;
        rotateLeft(k);
    }
    // Case 3: Node is left child → rotateRight
    parent.isRed = false;
    grandparent.isRed = true;
    rotateRight(grandparent);
}
} else {
    // Mirror case (parent is grandparent.right)
    RBNode uncle = grandparent.left;

    if (uncle != null && uncle.isRed) {
        grandparent.isRed = true;
        parent.isRed = false;
        uncle.isRed = false;
        k = grandparent;
    } else {
        if (k == parent.left) {
            k = parent;
            rotateRight(k);
        }
    }
}

```

```

        parent.isRed = false;

        grandparent.isRed = true;

        rotateLeft(grandparent);

    }

}

root.isRed = false;
}

```

Insert Function

```

void insert(int key) {

    RBNode node = new RBNode(key);

    root = bstInsert(root, node);

    fixInsert(node);

}

// BST insert
RBNode bstInsert(RBNode root, RBNode node) {

    if (root == null)

        return node;

    if (node.data < root.data) {

        root.left = bstInsert(root.left, node);

        root.left.parent = root;

    } else {

        root.right = bstInsert(root.right, node);

        root.right.parent = root;

    }

    return root;

}

```

Inorder Traversal (Print Sorted Tree)

```
void inorder(RBNode node) {  
    if (node != null) {  
        inorder(node.left);  
        System.out.print((node.isRed ? "R" : "B") + node.data + " ");  
        inorder(node.right);  
    }  
}
```

Red-Black Tree Interview Concepts

Concept	Asked In
Why use Red-Black instead of AVL?	System Design, Java Internals
Insert/Delete Logic	DSA rounds
Red-Black Tree vs AVL Tree	Deep understanding
Application in TreeMap / TreeSet	Java Internals

Red-Black Tree vs AVL Tree

Feature	AVL Tree	Red-Black Tree
Balance	Stricter	Looser
Rotations	More frequent	Fewer
Performance	Better for Search	Better for Insert/Delete
Use Case	DB Indexing	OS, Maps/Sets
Implementation	More complex	Moderately complex

Example Execution

```
public class RBTreeTest {  
    public static void main(String[] args) {  
        RedBlackTree rbt = new RedBlackTree();  
  
        int[] values = {10, 20, 30, 15, 25, 5, 1};  
        for (int v : values)  
            rbt.insert(v);  
  
        System.out.print("Inorder with colors: ");  
        rbt.inorder(rbt.root);  
    }  
}
```

Real-World Applications

- Self-balancing trees are used in database management systems to maintain sorted data and ensure quick insertion, deletion, and lookup.
- Operating systems use self-balancing trees in memory management and process scheduling to keep operations efficient and predictable.
- File systems utilize self-balancing trees to manage directories and file metadata for fast and organized access.
- Network routers use self-balancing trees to store and search routing information like IP addresses for optimized packet forwarding.
- Compilers use self-balancing trees to implement symbol tables for efficient management of variables, functions, and scopes.
- Real-time gaming engines use self-balancing trees for collision detection and spatial indexing of moving objects in a scene.

Good Conceptual Coding Problems

//1. Lowest Common Ancestor in a Binary Tree

```
class Solution {  
  
    public TreeNode lowestCommonAncestor(TreeNode root, TreeNode p, TreeNode q) {  
  
        if (root == null || root == p || root == q)  
            return root;  
  
        TreeNode left = lowestCommonAncestor(root.left, p, q);  
        TreeNode right = lowestCommonAncestor(root.right, p, q);  
  
        if (left != null && right != null)  
            return root;  
  
        return (left != null) ? left : right;  
    }  
}
```

//2. Diameter of Binary Tree

```
class Solution {  
  
    int max = 0;  
  
    public int diameterOfBinaryTree(TreeNode root) {  
        depth(root);  
        return max;  
    }  
  
    private int depth(TreeNode node) {  
        if (node == null) return 0;  
  
        int left = depth(node.left);  
        int right = depth(node.right);  
        max = Math.max(max, left + right);  
    }  
}
```

```

        return 1 + Math.max(left, right);
    }
}

```

//3. Serialize and Deserialize Binary Tree

```

public class Codec {

    // Encodes a tree to a single string.

    public String serialize(TreeNode root) {

        if (root == null) return "null,";

        return root.val + "," + serialize(root.left) + serialize(root.right);

    }


    // Decodes your encoded data to tree.

    int index = 0;

    public TreeNode deserialize(String data) {

        String[] nodes = data.split(",");

        return buildTree(nodes);

    }


    private TreeNode buildTree(String[] nodes) {

        if (nodes[index].equals("null")) {

            index++;

            return null;

        }

        TreeNode node = new TreeNode(Integer.parseInt(nodes[index++]));

        node.left = buildTree(nodes);

        node.right = buildTree(nodes);

        return node;

    }

}

```

//4. Validate Binary Search Tree

```
class Solution {  
  
    public boolean isValidBST(TreeNode root) {  
        return validate(root, Long.MIN_VALUE, Long.MAX_VALUE);  
    }  
  
    private boolean validate(TreeNode node, long min, long max) {  
        if (node == null) return true;  
        if (node.val <= min || node.val >= max) return false;  
  
        return validate(node.left, min, node.val) &&  
            validate(node.right, node.val, max);  
    }  
}
```

//5. Maximum Path Sum in Binary Tree

```
class Solution {  
  
    int maxSum = Integer.MIN_VALUE;  
  
    public int maxPathSum(TreeNode root) {  
        pathSum(root);  
        return maxSum;  
    }  
  
    private int pathSum(TreeNode node) {  
        if (node == null) return 0;  
  
        int left = Math.max(0, pathSum(node.left));  
        int right = Math.max(0, pathSum(node.right));  
  
        maxSum = Math.max(maxSum, left + right + node.val);  
        return node.val + Math.max(left, right);  
    }  
}
```



```

        return node.val + Math.max(left, right);
    }
}

```

//6. Zigzag Level Order Traversal of Binary Tree

```

import java.util.*;

class Solution {

    public List<List<Integer>> zigzagLevelOrder(TreeNode root) {

        List<List<Integer>> result = new ArrayList<>();

        if (root == null) return result;

        Queue<TreeNode> queue = new LinkedList<>();

        boolean leftToRight = true;

        queue.offer(root);

        while (!queue.isEmpty()) {

            int size = queue.size();

            LinkedList<Integer> level = new LinkedList<>(); // Use LinkedList for zigzag insertion

            for (int i = 0; i < size; i++) {

                TreeNode node = queue.poll();

                if (leftToRight)

                    level.addLast(node.val);

                else

                    level.addFirst(node.val);

                if (node.left != null)

                    queue.offer(node.left);
            }

            leftToRight = !leftToRight;

            result.add(level);
        }

        return result;
    }
}

```

```
        if (node.right != null)
            queue.offer(node.right);
    }

    result.add(level);

    leftToRight = !leftToRight; // Flip direction
}

return result;
}
}
```